

ÜBUNGEN

Allgemeine Hinweise

Wenngleich die nachfolgenden Übungen nicht direkt klausurrelevant sind, ist es dringend empfohlen sich mit ihnen in Hinblick auf die Klausur zu befassen.

Es handelt sich dabei weniger um Hausaufgaben im Sinne einer kontrollierten Zusatzleistung, sondern soll hauptsächlich dem Einüben der erlernten Inhalte dienen.

Sofern die einzelnen Schritte einer Aufgabe in nummerierter Reihenfolge angegeben sind, ist diese Ordnung bei der Bearbeitung einzuhalten.

Es bietet sich an, wenn man Aufgaben am Computer bearbeitet, die Lösung sofort im Internet zu suchen. Alle nachfolgenden Aufgaben sind damit sehr einfach zu lösen, allerdings sind während der Klausur keine internetfähigen Hilfsmittel erlaubt.

Aus diesem Grund sein gesagt, dass alle Aufgaben zunächst nur mit Hilfe des Skriptes probiert werden sollten. Sollte sich nach längerem Nachdenken kein Erfolg einstellen, können auch andere Mitglieder des Kurses gefragt werden.

Bei hartnäckigen Zweifeln hilft der Dozent gerne weiter.

IntelliJ IDE Shortcuts

Die IntelliJ IDE bietet viele beim Programmieren nützliche Komfortfunktionen an. Zu den wichtigsten und am häufigsten verwendeten gehören:

Funktion	Tastenkürzel	Beschreibung
Go To Hyperlink	Ctrl + B (Ctrl + Mausklick)	Gehe zu dem Codeabschnitt, an welchem das angeklickte Element (z.B. Methode, Variable) definiert ist.
Format Code	Ctrl + Alt + L	Formatiere den Code entsprechend des definierten Formatters (dieser kann pro Sprache individuell angepasst werden).
Rename (Refactor)	Shift + F6	Benenne das markierte Element (z.B. Variable, Methode, Klasse) um.
Organize Imports	Ctrl + Alt + O	Füge alle nötigen Imports hinzu und entferne unnötige.
Open Resource	2x Shift	Öffne ein Suchfeld um im gesamten Projekt nach etwas zu suchen.
Find	Ctrl + F	Öffne ein einfaches Finden Fenster, um in der aktuellen Datei etwas zu suchen.
Replace	Ctrl + R	Öffne ein einfaches Finden/Ersetzen Fenster.
Quick Outline	Ctrl + O	Zeige ein Popup-Menü mit allen Methoden/Variablen der aktuellen Klasse an.
Copy, Cut, Paste, Select All	Ctrl + C, Ctrl + X, Ctrl + V, Ctrl + A	Kopiere den markierten Bereich in die Zwischenablage. Schneide den markierten Bereich aus, in die Zwischenablage. Füge den in der Zwischenablage enthaltenen Bereich hier ein. Markiere alles.

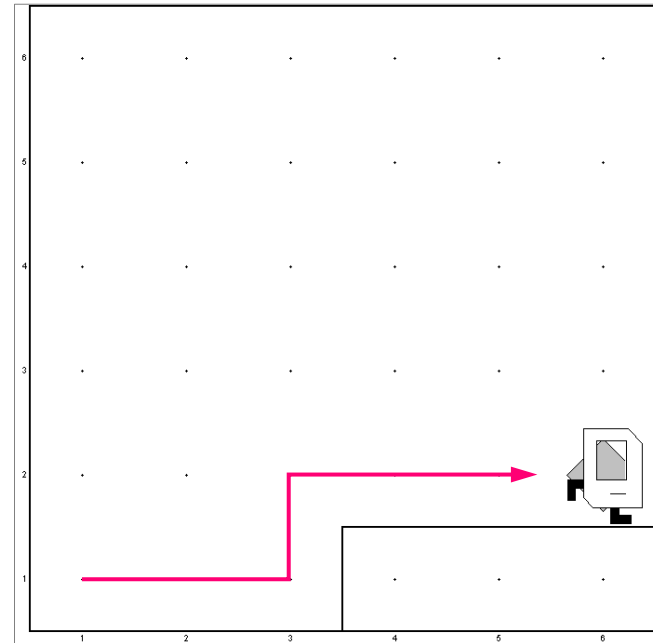
Karels Aufgaben

Aufgabe 1: BeeperCarryKarel

Es soll der Beeper aufgehoben und rechts auf dem Podest abgelegt werden.

```
1 import com.pgoellner.karel.*;
2
3 public class BeeperCarryKarel extends Karel {
4
5     public void run() {
6         move();
7         pickBeeper();
8         move();
9         turnLeft();
10        move();
11        turnLeft();
12        turnLeft();
13        turnLeft();
14        move();
15        move();
16        move();
17        putBeeper();
18    }
19
20 }
```

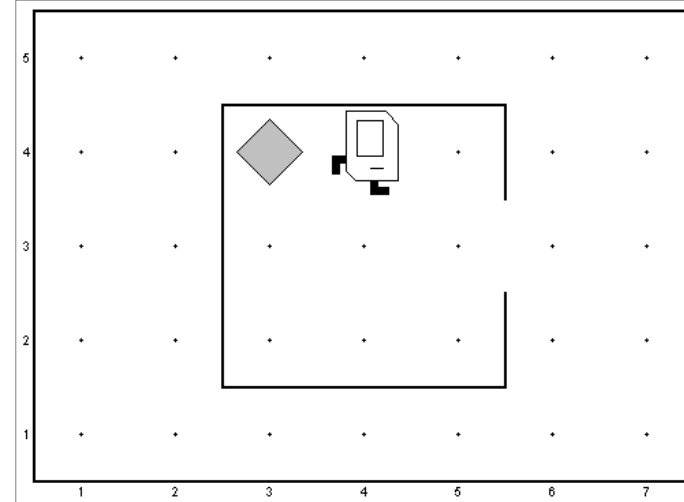
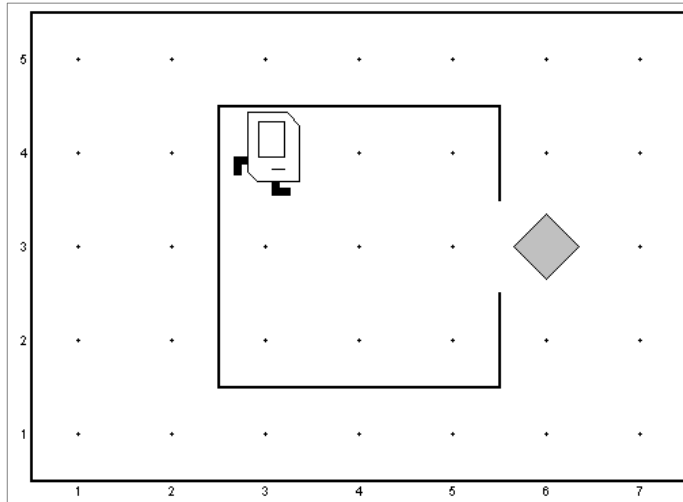
Der Java Quellcode



Erwartetes Ergebnis

Aufgabe 2: CollectNewspaperKarel

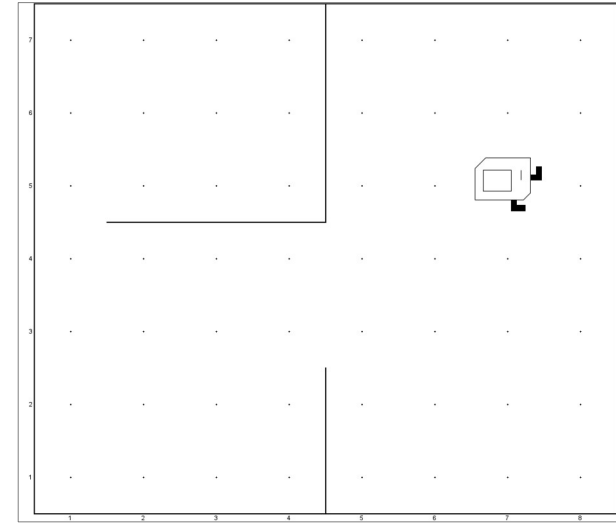
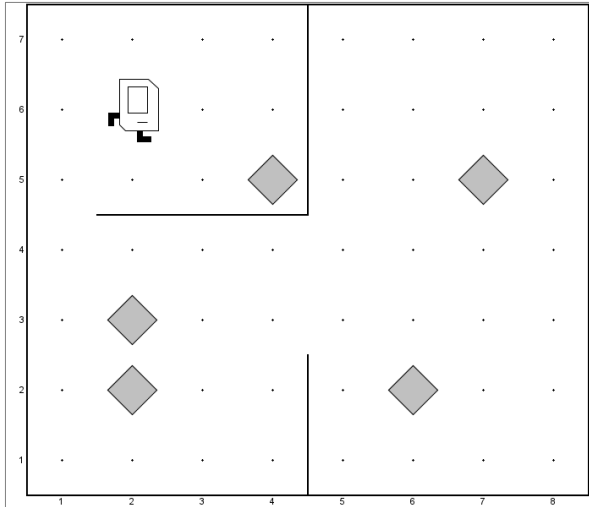
Der Beeper vor der Tür (*Zeitung*) soll abgeholt und an den Ausgangspunkt gelegt werden.



Erwartetes Ergebnis

Aufgabe 3: PartyCleanupKarel

Nach einer Party muss Karel seine Wohnung aufräumen (alle Beeper aufsammeln).

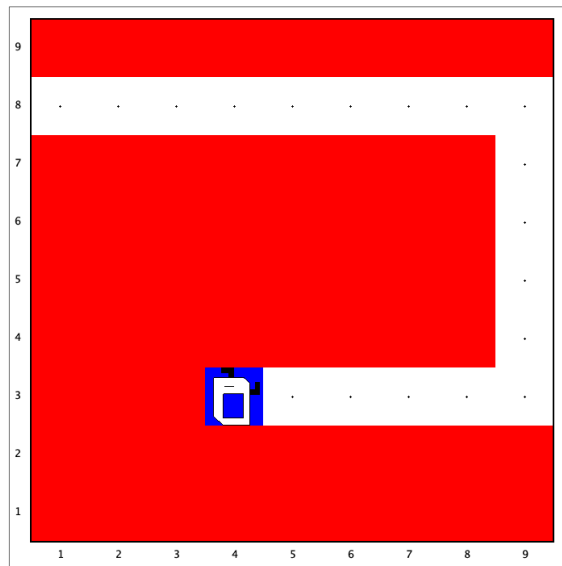


Erwartetes Ergebnis

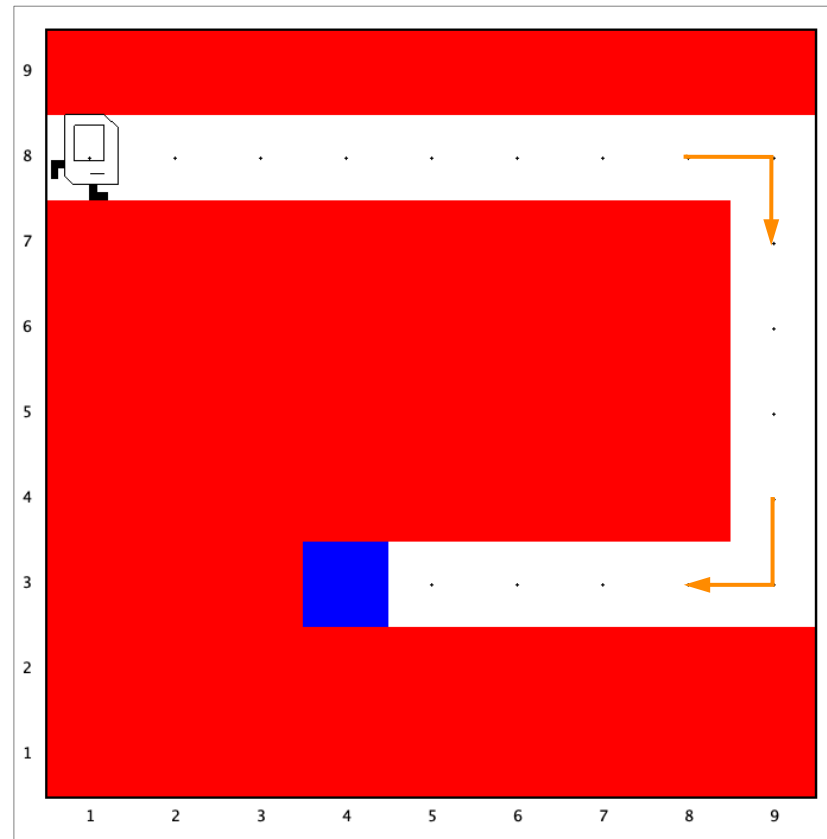
Aufgabe 4: FloorIsLavaKarel

Karel soll entlang des Pfades zum blauen Feld laufen und dort stehen bleiben.

Hinweis: Die roten Felder dürfen dabei nicht betreten werden!



Erwartetes Ergebnis



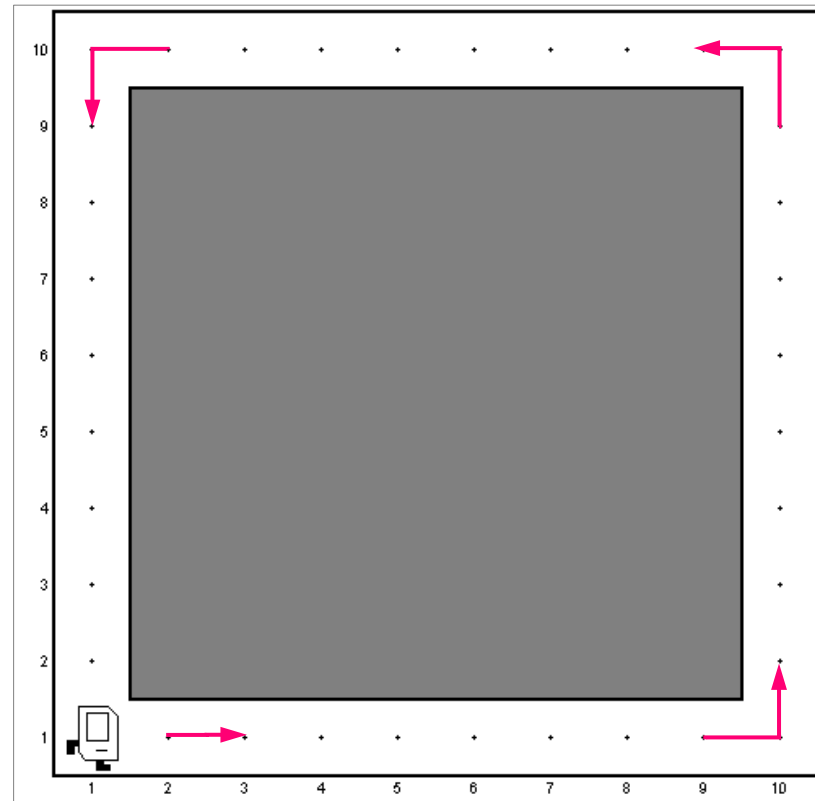
Aufgabe 5: SimpleRaceTrackKarel

Es sollen unendlich viele Runden auf der einfachen Rennstrecke gedreht werden.

Hinweis: Es sollte eine Endlosschleife verwendet werden.

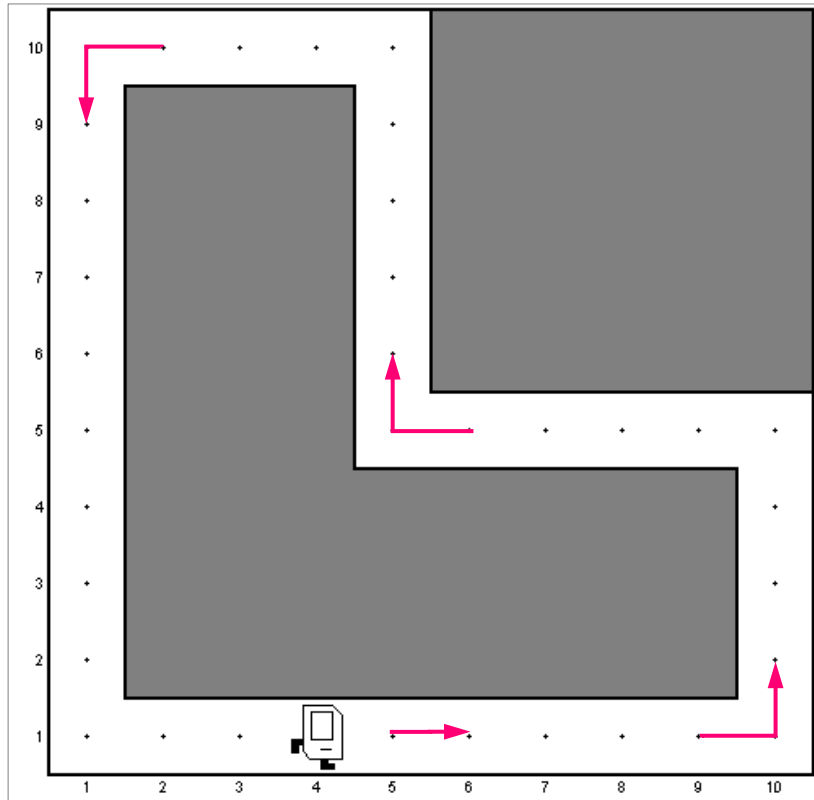
Syntax:

```
while(true) {  
    /* ... */  
}
```



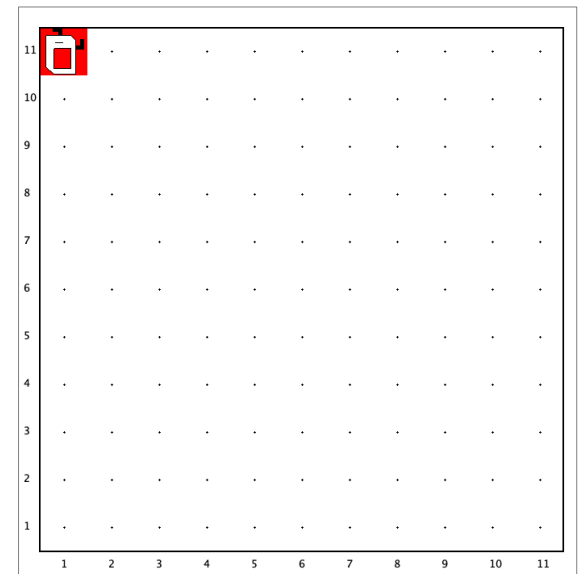
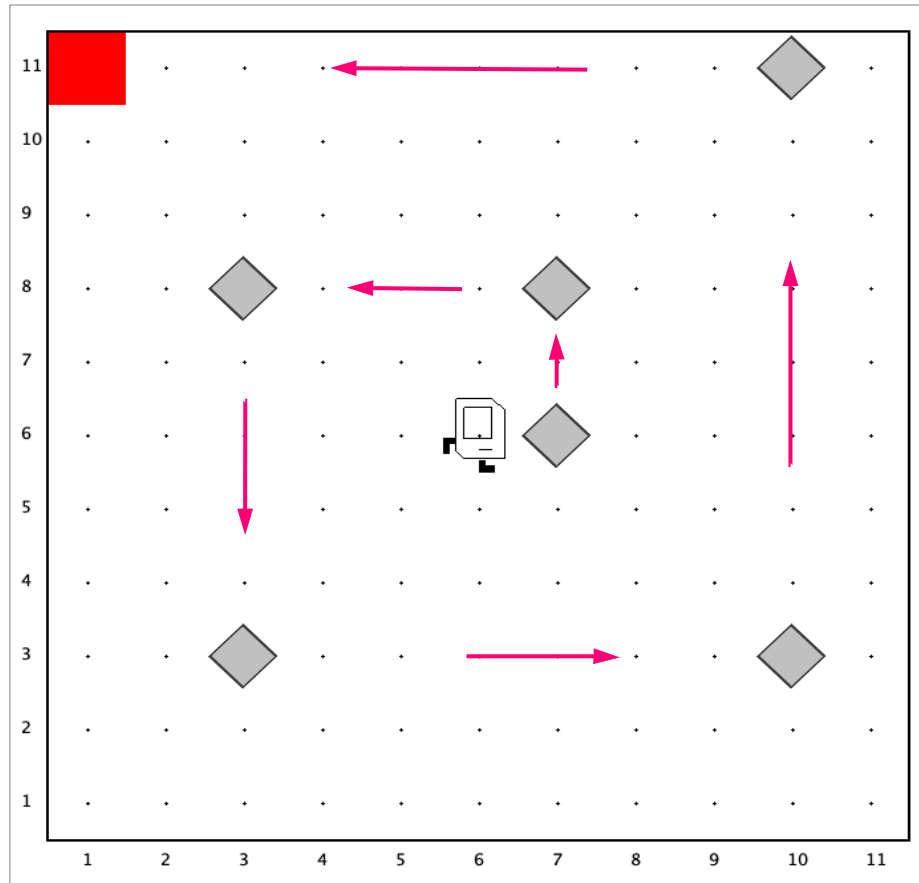
Aufgabe 6: AdvancedRaceTrackKarel

Es sollen unendlich viele Runden auf der erweiterten Rennstrecke gedreht werden.



Aufgabe 7: SpiralRunKarel

Die Beeper, die in einem Spiralenmuster ausliegen, sollen alle aufgesammelt werden.
Versuche die Aufgabe mit so wenig Anweisungen wie möglich zu lösen.



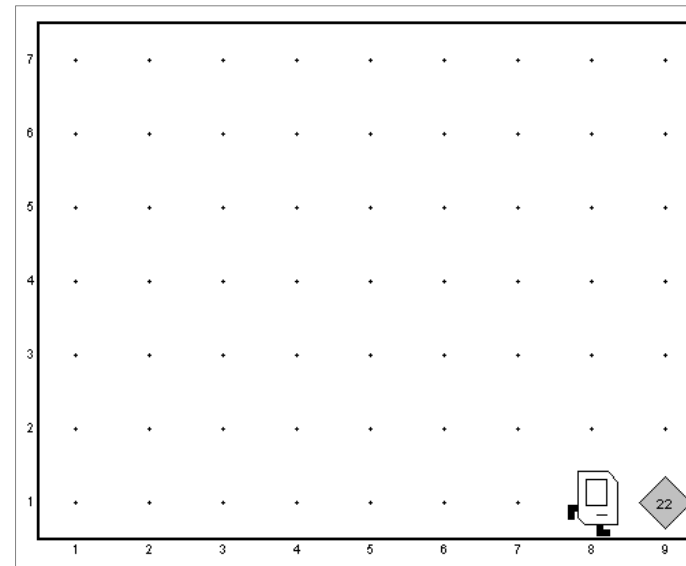
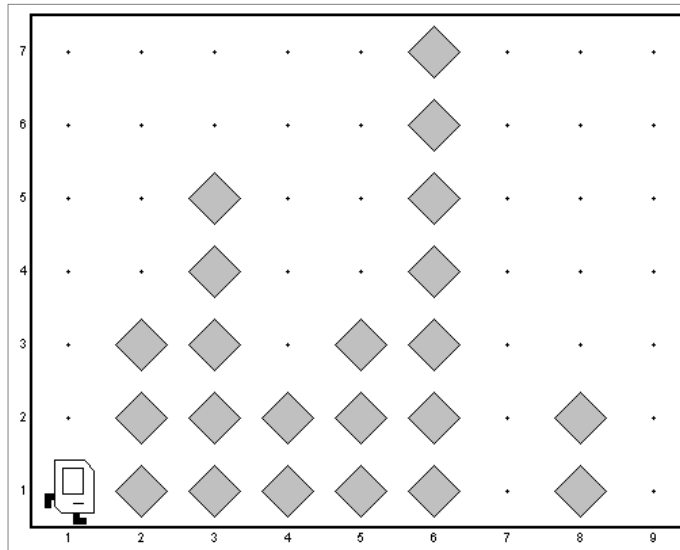
Erwartetes Ergebnis

Aufgabe 8: BeeperCollectingKarel

Es sollen alle Beeper auf dem Spielfeld aufgesammelt werden und in der unteren rechten Ecke abgelegt werden.

Wichtig: Das Programm soll so geschrieben sein, dass es auch mit einer anderen Verteilung der Beeper oder einer anderen Spielfeldgröße funktioniert.

Hinweise: Um mehrere Beeper abzulegen, muss `putBeeper()` mehrmals aufgerufen werden.
Karel merkt sich nicht von alleine wie viele Beeper aufgesammelt worden sind.

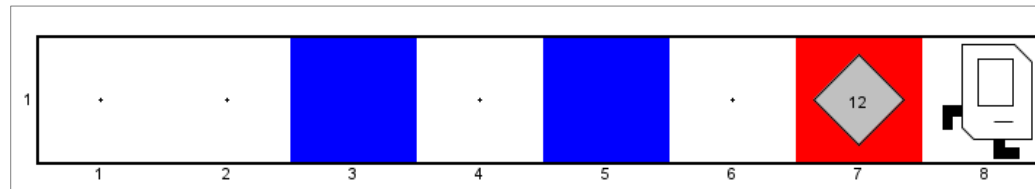
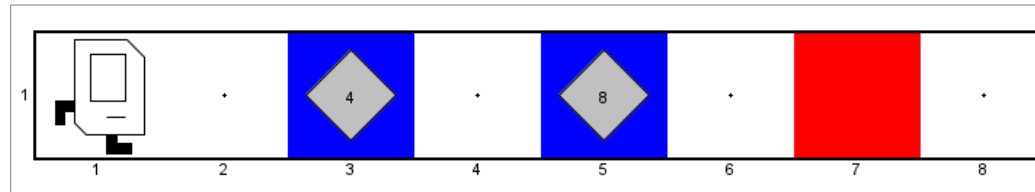


Erwartetes Ergebnis

Aufgabe 9: CalculatorKarel

Die Beeper auf den blauen Feldern sollen aufgesammelt und auf dem roten Feld abgelegt werden. Die Anzahl der abgelegten Beeper soll gleich der Summe der Beeper auf den blauen Feldern sein.

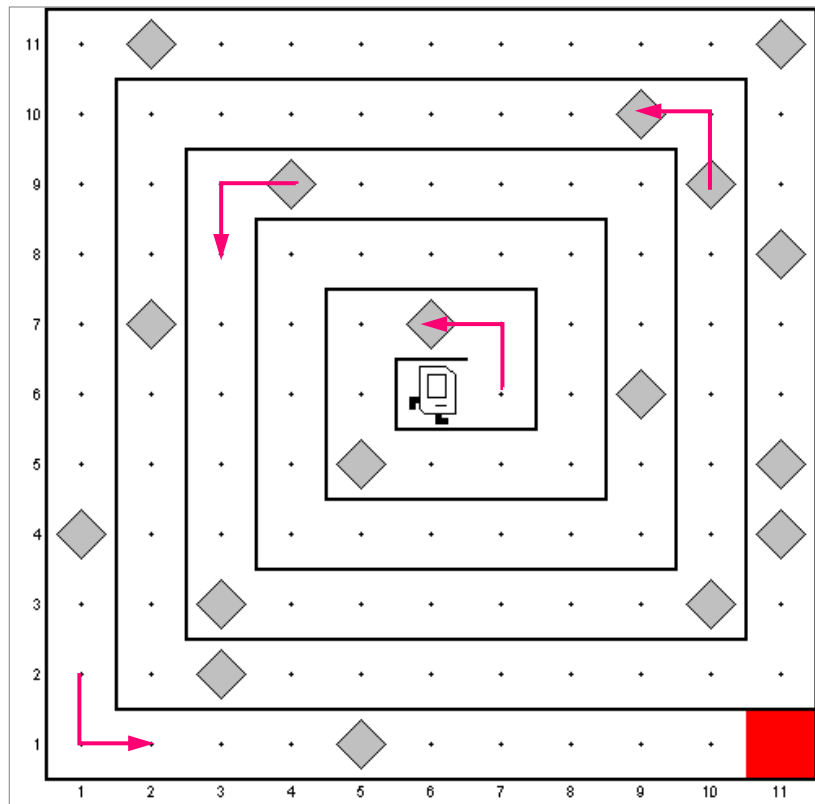
Wichtig: Das Programm soll so geschrieben sein, dass es auch mit unterschiedlich vielen Beepern auf den blauen Feldern funktioniert.



Erwartetes Ergebnis

Aufgabe 10: SprrialEscapeKarel

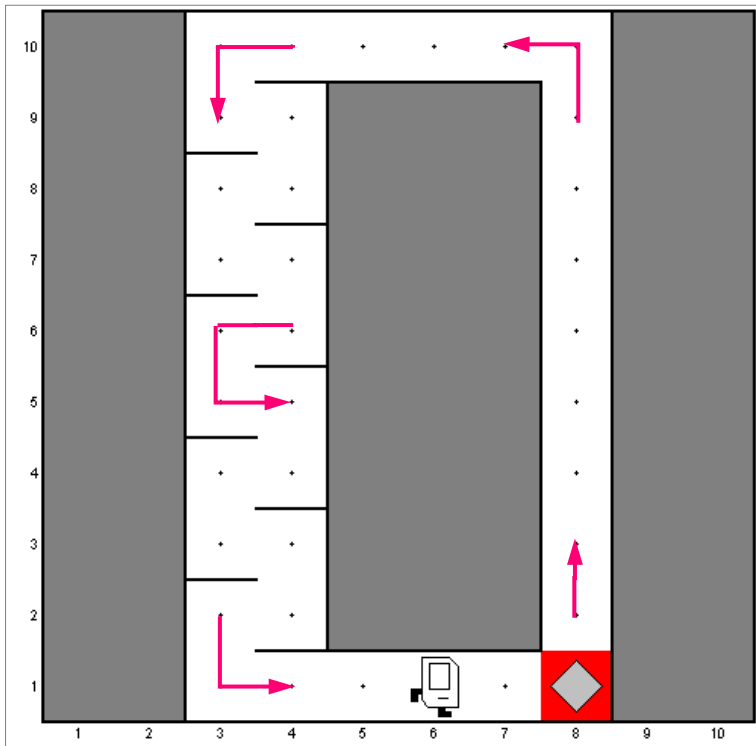
Karel soll den Weg aus dem Labyrinth zum roten Feld finden und dabei alle auf dem Spielfeld vorhandenen Beeper aufsammeln. Alle aufgesammelten Beeper sollen anschließend auf das rote Feld gelegt werden.



Aufgabe 11: SlalomKarel

Karel soll unendlich viele Runden auf der Slalom-Strecke drehen. Bei jeder Überquerung des roten Feldes soll dort ein Beeper abgelegt werden.

Hinweis: Es ist nicht die Farbe eines Feldes entscheidend, sondern ob dort ein Beeper liegt

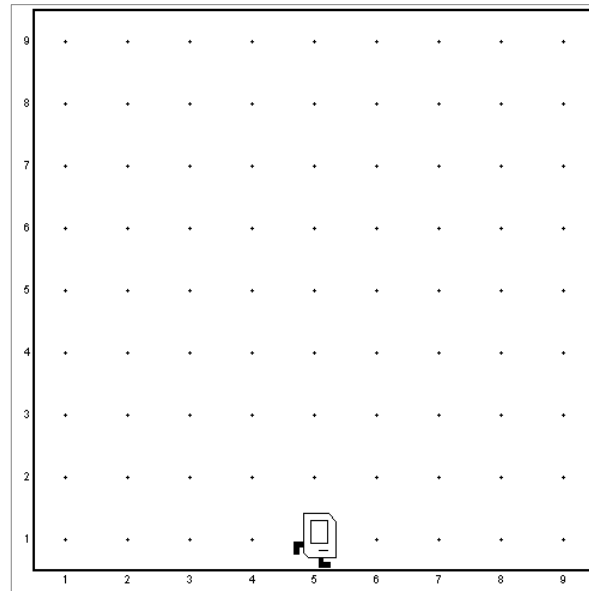
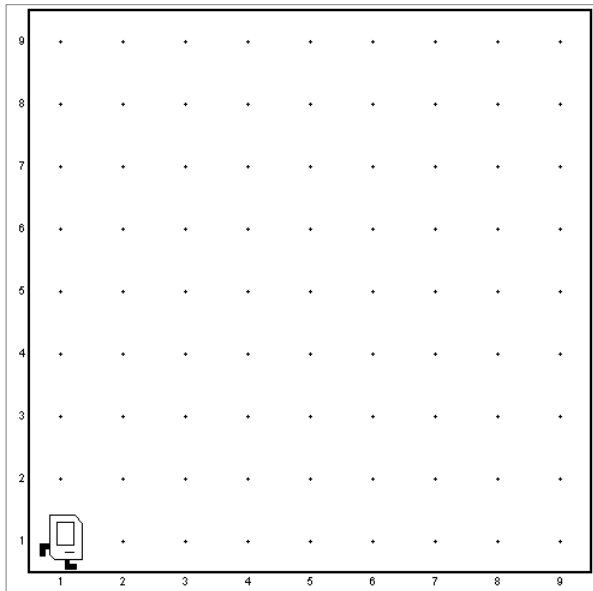


Zusatzaufgabe: Immer wenn Karel auf dem roten Feld steht, soll er seine Laufrichtung wechseln.

Aufgabe 12: MidpointFindingKarel

Karel soll genau die Mitte der untersten Zeile finden und zu ihr gehen.

Hinweis: Das Programm soll so geschrieben sein, dass Karel die Mitte findet, egal wie viele Spalten das Spielfeld hat. Bei gerader Spaltenzahl ist eine der beiden mittleren Spalten ok.



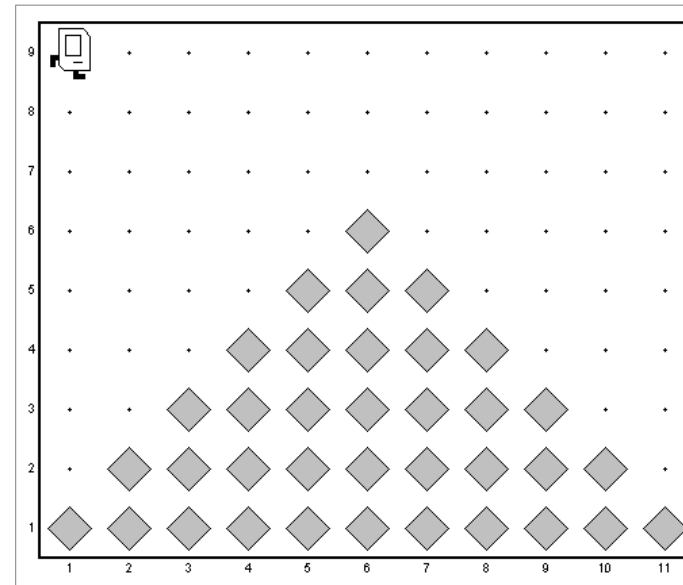
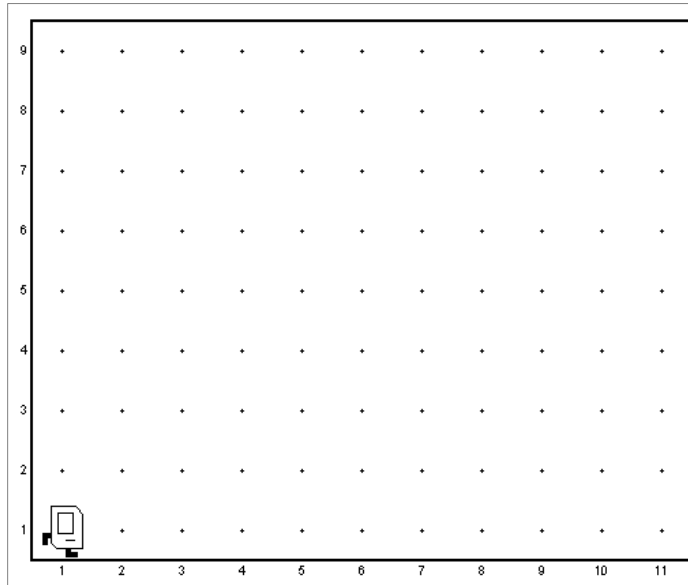
Erwartetes Ergebnis

Zusatzaufgabe: Zusätzlich soll neben der horizontalen Mitte auch die vertikale gefunden werden.

Aufgabe 13: PyramidKarel

Es soll eine Pyramide gebaut werden, die genau vom linken zum rechten Spielfeldrand reicht.

Hinweis: Das Programm soll so geschrieben sein, dass die Pyramide mindestens für die folgenden Spielfeldgrößen korrekt erstellt wird (BxH): 11x10, 15x10, 5x15, 3x5.



Erwartetes Ergebnis

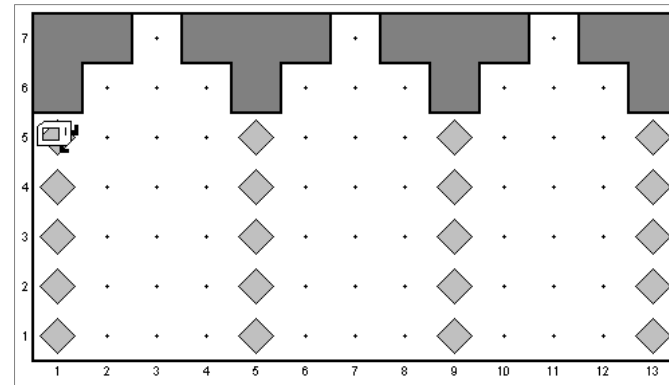
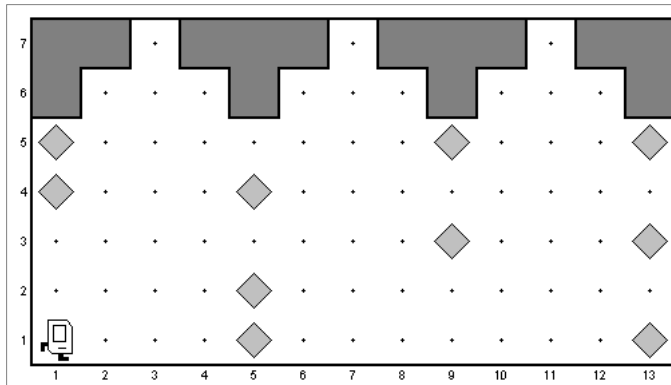
Aufgabe 14: StoneMasonKarel

Die vier Säulen sollen wieder aufgebaut werden, indem an den fehlenden Stellen jeweils ein Beeper platziert wird

Hinweise: Das Gesamtproblem lässt sich in zwei Teilprobleme zerlegen, die mit separaten Methoden gelöst werden können: `makeColumn()` und `moveToNextColumn()`

Für das Verständnis ist es einfacher, wenn `makeColumn()` so implementiert ist, dass Karel nach dem Methodenaufruf am selben Platz steht (und in dieselbe Richtung blickt) wie vor dem Methodenaufruf.

Der Abstand der Säulen und die Abmessungen des Spielfelds können als konstant angenommen werden.



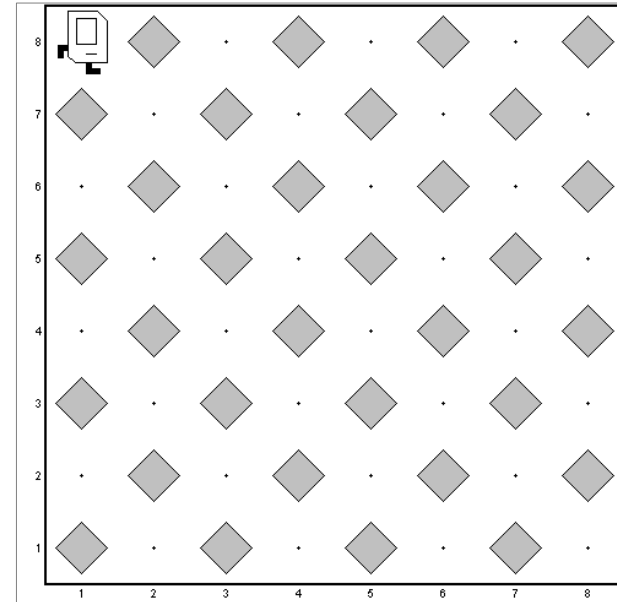
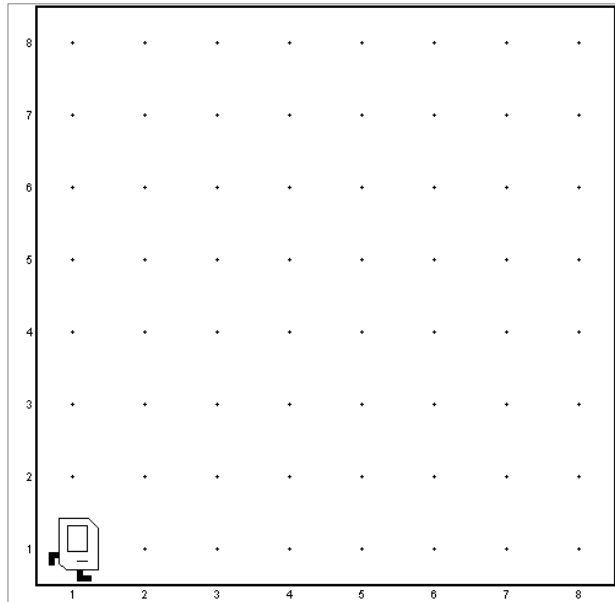
Erwartetes Ergebnis

Zusatzaufgabe: Karel soll das Muster mit möglichst wenig Schritten erstellt.

Aufgabe 15: CheckerboardKarel

Karel soll mit Beepern ein Schachbrettmuster auf der gesamten Spielfläche auslegen

Hinweis: Das Programm soll so geschrieben sein, dass es auch mit einer anderen Spielfeldgröße funktioniert.



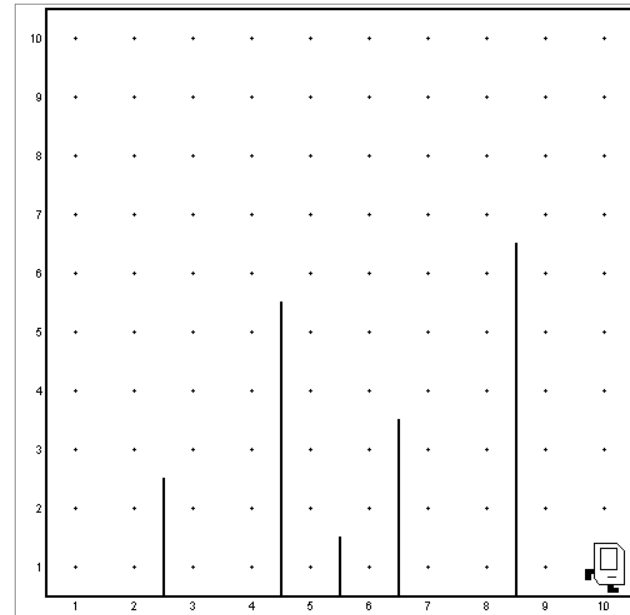
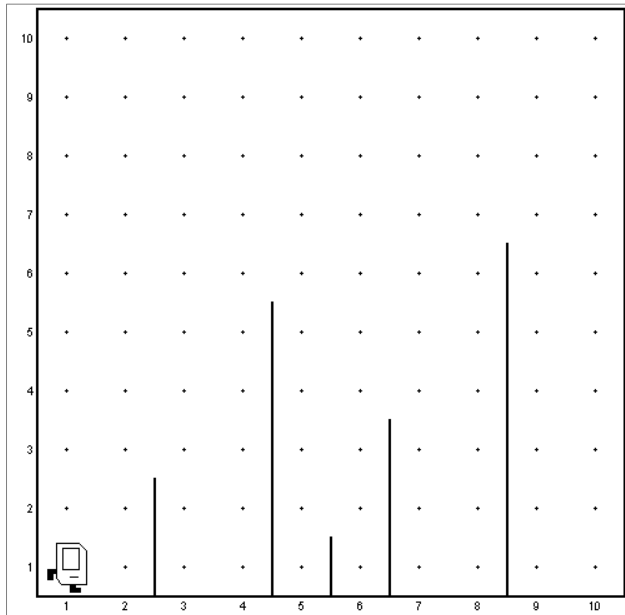
Erwartetes Ergebnis

Zusatzaufgabe: Karel soll das Muster mit möglichst wenig Schritten erstellt.

Aufgabe 16: HurdleJumpingKarel

Karel soll jede Hürde einzeln überspringen und so die untere rechte Ecke erreichen.

Hinweis: Das Programm soll so geschrieben sein, dass es auch mit einem anderen Aufbau der Hürden oder einer anderen Spielfeldgröße funktioniert. Daher muss Karel den Spielfeldrand zuverlässig erkennen können.



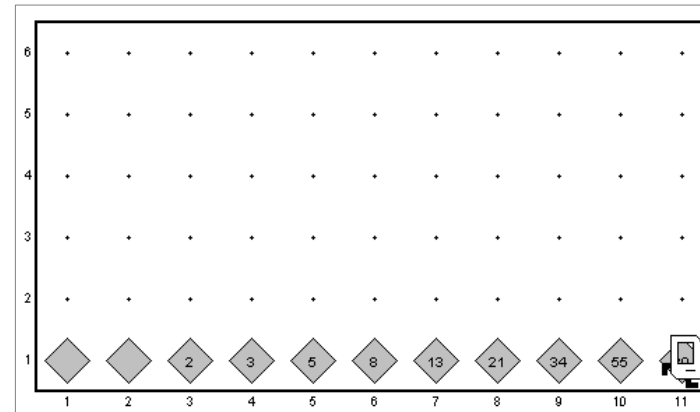
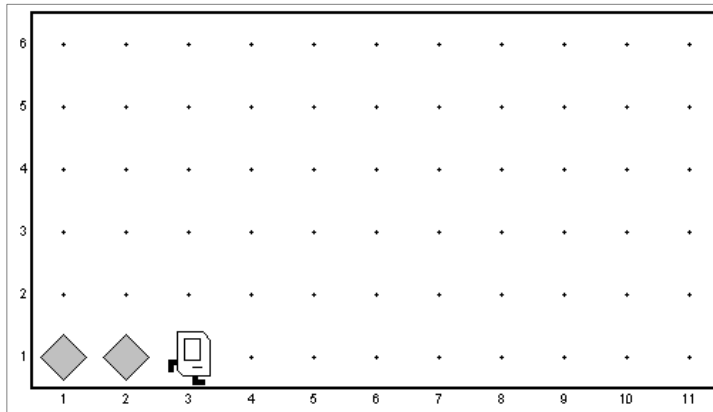
Erwartetes Ergebnis

Aufgabe 17: FibonacciKarel

Es soll mit Beepern eine berechnete [Fibonacci-Folge](#) erstellt werden, die vom linken bis zum rechten Spielfeldrand reicht.

Hinweise: Die Aufgabe kann mittels eines [iterativen](#) oder eines [rekursiven](#) Algorithmus gelöst werden.

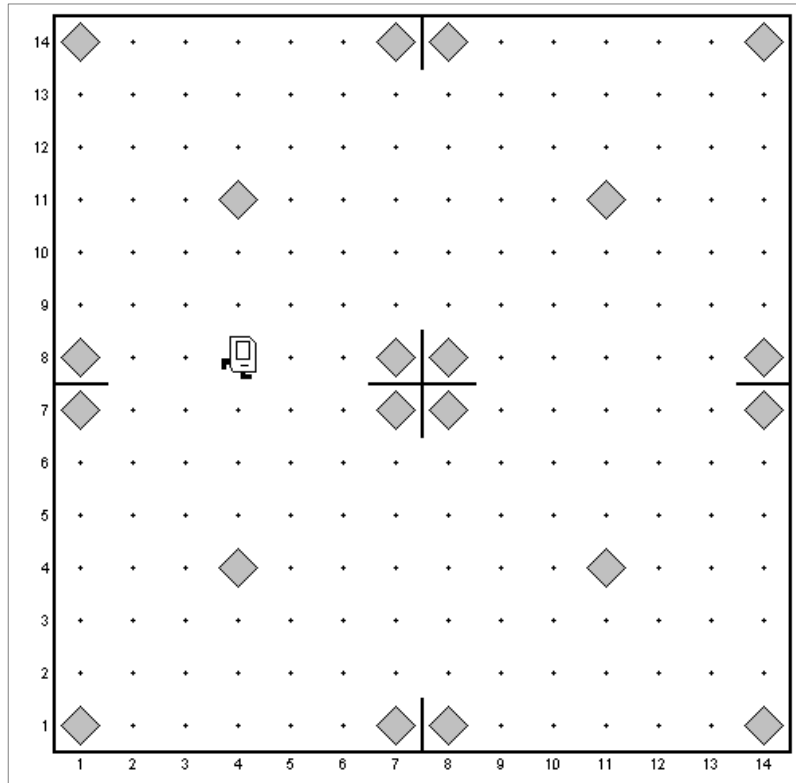
Das Programm soll so geschrieben sein, dass es auch mit einer anderen Spielfeldgröße funktioniert.



Erwartetes Ergebnis

Aufgabe 18: DeflectionKarel

Karel soll sich unentwegt geradeaus über das Spielfeld bewegen. Immer wenn er eine Wand trifft, kehrt er um (180°). Bei jedem 10. Schritt legt Karel einen Beeper (außer dort liegt schon einer). Immer wenn Karel einen Beeper trifft, hebt er diesen auf, dreht sich zufällig `Math.random()` nach rechts oder links (90°) und geht dann weiter.



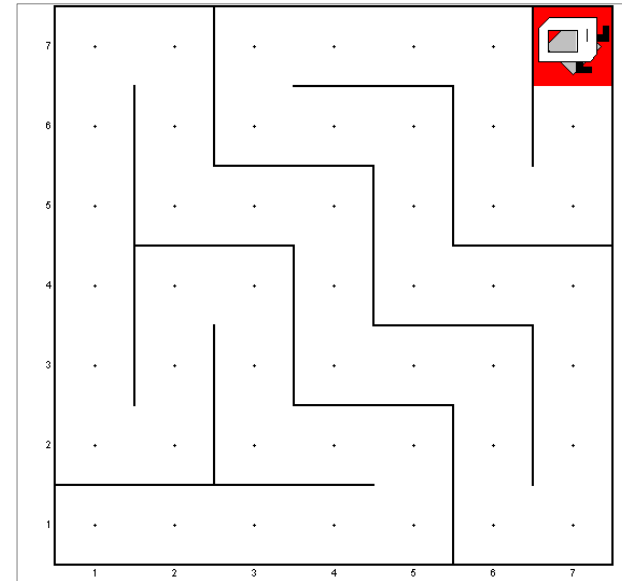
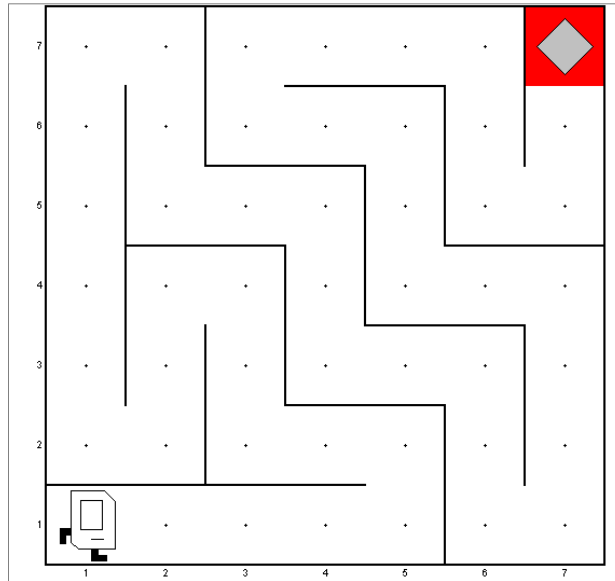
Aufgabe 19: LabyrinthKarel

Karel soll den Weg durch das Labyrinth, zum Beeper finden.

Für Einsteigende: Man kann auch erstmal mit dem
TinyLabyrinthKarel anfangen.

Für Fortgeschrittene: Das Programm soll auch die folgenden
komplexen Levels/Worlds lösen:

- MediumLabyrinthKarel
- LargeLabyrinthKarel
- MadLabyrinthKarel

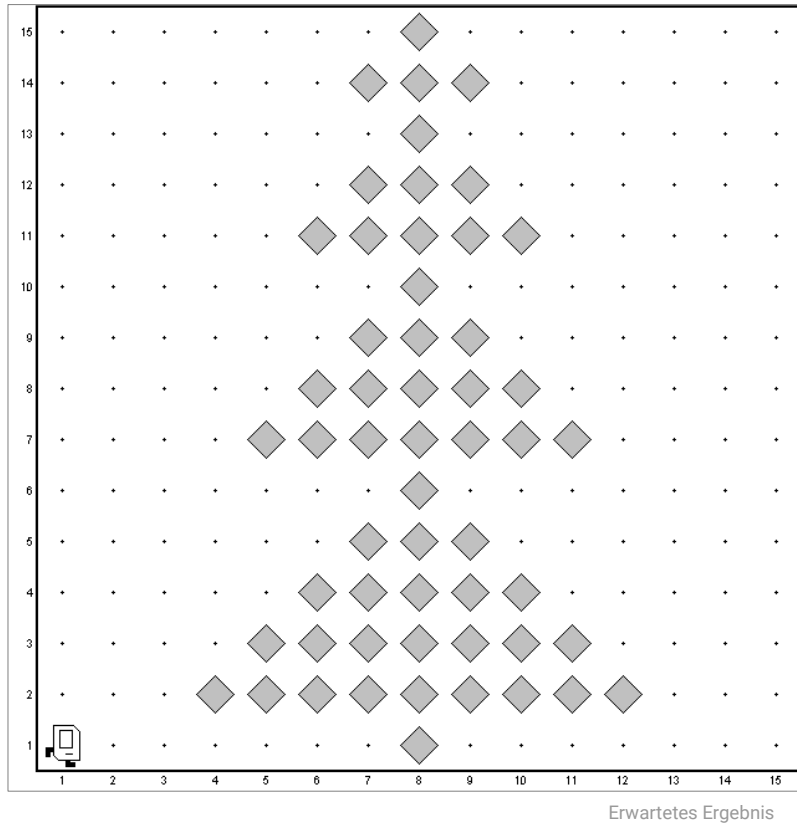


Erwartetes Ergebnis

Aufgabe 20: ChristmasTreeKarel

Es soll ein großer Weihnachtsbaum aus Beepern gelegt werden.

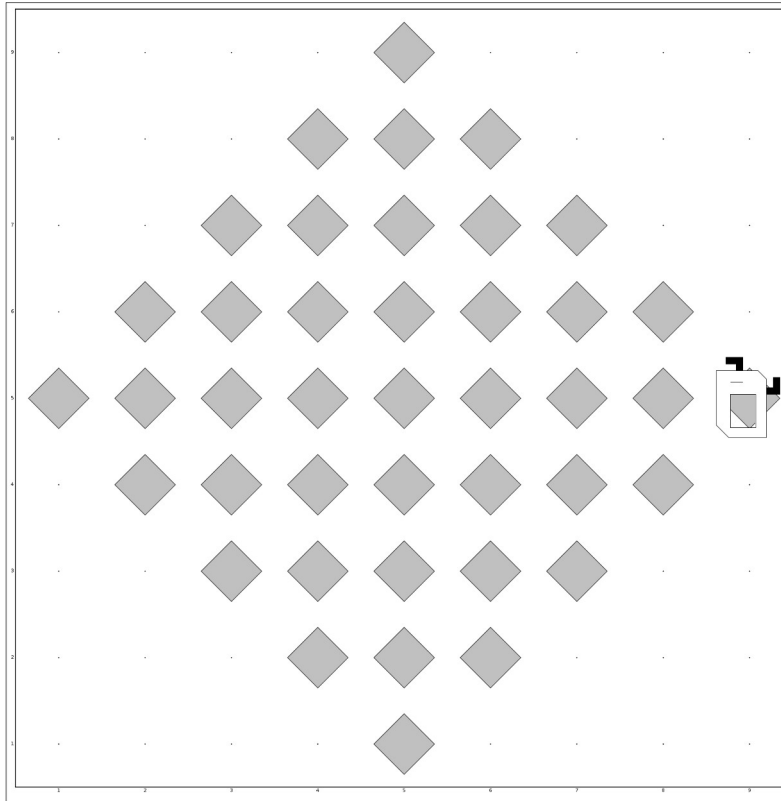
Hinweis: Das freie Spielfeld hat eine Größe von 15x15.



Zusatzaufgabe: Der Code soll unverändert mit jeder beliebigen Spielfeldgröße funktionieren und den Baum entsprechend skalieren.

Aufgabe 21: DiamondKarel

Gegeben ist ein Programm, das bei einem quadratischen Feld (Größe von 9x9) eine Raute aus Beepern legt. Allerdings ist der Code sehr unübersichtlich und schwer zu verstehen. Daher soll er so verändert werden, dass er seine Funktionalität behält, aber eine bessere Struktur erhält.



Zusatzaufgabe: Ändere den Code anschließend so ab, dass er auch mit nicht-quadratischen Feldern funktioniert.

Eigenständige Java-Projekte

FizzBuzz

FizzBuzz ist eine einfache Einstiegs-Aufgabe, die gelegentlich bei Einstellungsgesprächen gestellt wird. Sie dient hier der Einübung von Verzweigungen und bool'schen Ausdrücken.

Ein FizzBuzz-Programm gibt eine Zahlenfolge natürlicher Zahlen aus, jedoch werden alle Zahlen, die durch 3 teilbar sind, durch Fizz ersetzt, alle durch 5 teilbaren Zahlen durch Buzz, und alle Zahlen, die sowohl durch 3 als auch durch 5 teilbar sind, durch FizzBuzz.

Die ersten 15 Ausgaben eines FizzBuzz-Programms sehen wie folgt aus:
1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz



```
Run FizzBuzzRunner x
/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home/bin/java ...
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
```

Beispielausgabe

1. Erzeuge die Klasse **FizzBuzzRunner**. Sie hat keine Attribute.
2. Erstelle eine Methode **String fizzBuzz(int number)**. In ihr soll die übergebene Zahl **number** geprüft werden.
 - Ist die Zahl durch 3 teilbar, soll **"Fizz"** zurückgegeben werden
 - Ist die Zahl durch 5 teilbar, soll **"Buzz"** zurückgegeben werden
 - Ist die Zahl durch 3 und durch 5 teilbar, soll **"FizzBuzz"** zurückgegeben werden
 - In allen anderen Fällen soll die Zahl als String zurückgegeben werden (**" " + number**)
3. Erstelle eine Mainmethode, die in einer Schleife **fizzBuzz()** mit aufsteigenden Zahlen aufruft (z.B. 1 - 25).

Ein paar Beispiele für das Verhalten der fizzBuzz-Methode:

Eingabe	Ausgabe
1	"1"
2	"2"
3	"Fizz"
6	"Fizz"
5	"Buzz"
15	"FizzBuzz"

Vollkommene Zahlen

In der Mathematik ist eine Zahl dann als vollkommen definiert, wenn sie gleich der Summe aller ihrer Teiler (außer sich selbst) ist. Die beiden kleinsten vollkommenen Zahlen sind:

$$\begin{array}{lcl} 6 & = & 1 + 2 + 3 \\ 28 & = & 1 + 2 + 4 + 7 + 14 \end{array}$$

Es soll eine Methode hierzu entwickelt werden, die prüft, ob eine übergebene Zahl vollkommen ist.

1. Erzeuge Klasse **PerfectNumberChecker**. Sie hat keine Attribute.
2. Erstelle eine Methode **boolean isPerfectNumber(int number)**. Diese Methode soll **true** zurückliefern, wenn die übergebene Zahl **number** eine vollkommene Zahl ist, und andernfalls **false**.
3. Dekлариere eine **int** Variable **sum**, welche als Zwischenergebnis in der Methode dienen soll.
4. Erstelle eine Schleife, die die Zahlen von **1** bis **number-1** durchläuft (eine **for**-Schleife eignet sich hier am besten).
5. Bei jeder Schleifen-Iteration, prüfe, ob der Counter ein Teiler von **number** ist (hier wird der Module-Operator benötigt). Falls das der Fall ist, addiere den Counter zu **sum**.
6. Nach der Schleife, prüfe, ob **sum** gleich **number** ist. Wenn ja, gib **true** zurück, ansonsten **false**.

Schaltjahre

Bekannterweise decken sich ein Kalenderjahr von 365 Tagen und ein astronomisches Jahr nicht exakt. Aus diesem Grund wurde im Julianischen Kalender Model jedes 4. Jahr ein Schaltjahr eingefügt mit einem weiteren Tag (dem 29. Februar). Das wurde dann im Gregorianischen Kalender noch weiter verbessert.

Diese Übung soll eine Methode implementieren, die in der Lage ist anhand eines übergebenen Jahres zu bestimmen, ob dieses ein Schaltjahr ist oder nicht. Die Methodensignatur soll in etwa so aussehen: `boolean isLeapYear(int year)`

Die Ermittlung soll nach den folgenden Regeln erfolgen:

- Ist ein Jahr durch 400 teilbar, dann ist es ein Schaltjahr
- Ist ein Jahr durch 100 aber nicht durch 400 teilbar, dann ist es kein Schaltjahr
- Ist ein Jahr durch 4 aber nicht durch 100 teilbar, dann ist es ein Schaltjahr
- In allen anderen Fällen ist das Jahr kein Schaltjahr

Versuche diese Aufgabe ohne weitere Hilfestellungen umzusetzen.

SkyMasters

SkyMasters bildet ein [Supertrumpf](#)-Kartenspiel nach.

1. Erzeuge die Klasse **Airplane**. Sie hat das String-Attribut **name** sowie die double-Attribute **maxSpeed** (km/h), **maxAltitude** (m) und **price** (USD). Erzeuge die zugehörigen Akzessoren sowie den Konstruktor und die **toString**-Methode.
2. Erzeuge die Klasse **Sky**. In ihrer Main-Methode, erzeuge mindestens 2 verschiedene **Airplane**-Objekte, mit jeweils unterschiedlichen Attributen (z.B. [SR-71 Blackbird](#), [NA-X15](#), [An-225](#), [777-Worldliner](#), [B2-Spirit](#), [F22-Raptor](#)).
3. Vergleiche zwei **Airplanes** anhand eines fest vorgegebenen double-Attributs (z.B. **maxSpeed**) und gebe auf der Konsole aus, welches Flugzeug den Vergleich gewonnen hat und warum (größere Zahl schlägt kleinere).
4. Im nächsten Schritt soll der Zufall entscheiden, welches der drei double-Attribute für den Vergleich herangezogen wird. Kombiniere hierzu eine Zufallszahl aus **Math.random()** mit einer Verzweigung.
5. Welche Objekte mit einander verglichen werden, soll danach ebenfalls vom Zufall bestimmt werden.
6. Füge dem Programm weitere Klassen von Flugobjekten hinzu, z.B.: **Helicopter**, **Airship**, **Rocket**, **UFO**.
Alle Flugobjekt-Klassen sollen von der Mutterklasse **FlyingMachine** erben. Jede der Unterklassen muss die Methode **fly()** implementieren, welche auf der Konsole ausgibt, auf welchem Prinzip das Flugobjekt basiert (z.B. Tragflächen, Rotor, Aerostatik, Rückstoß, Levitation). Rufe diese Methode immer bei beiden ausgewählten Objekten aus.
7. Erzeuge in Sky entsprechende Objekte, z.B. [S-64 Skycrane](#) (**Helicopter**), [Eurocopter X3](#) (**Helicopter**), [Hindenburg](#) (**Airship**), [Zeppelin NT](#) (**Airship**), [Saturn V](#) (**Rocket**), [Falcon 9](#) (**Rocket**), [I-Day Attacker](#) (**UFO**), [Cylon Raider](#) (**UFO**).
8. Erweitert: Zeige alle Attribute eines zufällig gewählten Flugobjekts (A) auf der Konsole an. Nun lasse auswählen, welches Attribut für den Vergleich mit dem anderen (bis dahin nicht gezeigten) Flugobjekt (B) verwendet werden soll. Für das Lesen von der Konsole siehe das entsprechende Kapitel in Kapitel 4. Nachdem das Attribut ausgewählt wurde, werden A und B anhand des gewählten Attributs verglichen. Gewinnt A gegen B, erhält man einen Punkt. Andernfalls verliert man einen Punkt. Danach startet eine neue Runde (Schleifen-Wiederholung). Der aktuelle Punktestand wird in jeder Runde auf der Konsole angezeigt. Sobald man 3 Punkte erreicht, ist das Spiel gewonnen (das Programm wird beendet).
9. Erweitert: Füge dem weitere double-Attribute hinzu, z.B.: **maxRange** (km), **weight** (kg), **length** (m), **power** (kW)

Arrays

Arrays sind ein wichtiger Baustein der Java-Grundkenntnisse.
Nachfolgend finden sich mehrere kleine Übungen, um den Umgang mit
Arrays einzuüben. Erstelle hierzu eine eigenständige Klasse **ArrayExercises**.

Hinweis: Bei dieser Übung eignet sich in den meisten Punkten ein TDD-Ansatz

1. Die Methode **head(int[] intArray)** soll das erste Element des
übergebenen Arrays zurückgeben

2. Die Methode **last(int[] intArray)** soll das letzte Element des
übergebenen Arrays zurückgeben

3. Die Methode **printArray(int[] intArray)** soll die Werte eines
übergebenen Arrays auf der Konsole ausgeben

Zusatzübung: Die Ausgabe soll so aussehen:

```
<terminated> ArrayExercises  
[0, 5, 7, 11, 6]
```

4. Die Methode **doubleAll(int[] intArray)** soll alle Werte eines
übergebenen Arrays verdoppeln

5. Die Methode **printEveryTwo(int[] intArray)** soll nur jede zweite Stelle
auf der Konsole ausgeben

6. Die Methode **copyArray(int[] intArray)** soll das übergebene Array
kopieren und die Kopie zurückgeben

7. Die Methode **sum(int[] intArray)** soll alle Werte des übergebenen Arrays
addieren und das Ergebnis zurückgeben

8. Die Methode **init(int[] intArray)** soll alle Werte des übergebenen
Arrays bis auf den letzten in ein neues Array kopieren

```
[1, 2, 3]
```

9. Die Methode **tail(int[] intArray)** soll alle Werte des übergebenen
Arrays bis auf den ersten in ein neues Array kopieren

```
[1, 2, 3]
```

10. Die Methode **join(int[] array1, int[] array2)** soll die übergebenen
Arrays zu einem neuen Array zusammenfügen

array1

```
[1, 2, 3]
```

array2

```
[4, 5]
```

erwartetes Ergebnis

```
[1, 2, 3, 4, 5]
```

HelloRecursion

HelloRecursion soll einen einfachen rekursiven Algorithmus umsetzen.

1. Erzeuge die Klasse **HelloRecursion**, mit einer Main-Methode.
2. Darin, erstelle die Methode `void repeat(int repetitions)`, welche von der Main-Methode aufgerufen wird.
3. Die Methode soll von der angegebenen Zahl repetitions bis auf 0 herunterzählen und bei jedem Schritt den aktuellen Zähler auf der Konsole ausgeben: „Hello 10“, „Hello 9“, [...], „Hello 0“.
4. Teste diese iterative Variante mit `repetitions=100`.
5. Nun setze die Methode `repeat()` rekursiv um, d.h. ohne Verwendung einer Schleife. Hierbei muss sich `repeat()` selbst aufrufen sofern (`if`) der Zähler (`repetitions`) größer als 0 ist. Sollte die Abbruchbedingung nicht korrekt definiert sein, ruft sich die Methode endlos selbst auf, bis das Programm mit einem **StackOverflowError** abstürzt.
6. Sobald die rekursive Variante funktioniert, versuche es mit größeren Werten von repetitions: Erst 1000, dann 5000, dann 10000. Sollte das Programm bei einem der höheren Werte abstürzen, wurde wahrscheinlich die maximale Stack-Size der JVM überschritten. Als Hintergrund siehe dazu den Abschnitt „JVM Stack und Heap“ des nächsten Kapitels.
7. Anstatt herunterzuzählen, zähle jetzt von 1 bis zur angegebenen Zahl (100) hoch. Hierzu musst ggf. die Methode anpassen werden zu `repeat(int currentRepetition, int maxRepetitions)`.

Rekursive Arithmetik

Die Denkweise für rekursive Algorithmen soll weiter vertieft werden, indem die einfachen arithmetischen Operatoren (+,-,/,*) rekursiv implementiert werden. Hierbei sollen nur natürliche Zahlen verwendet werden.

1. Erzeuge die Klasse **RecursiveArithmetics**, mit einer Main-Methode.
2. Implementiere eine Methode **int add(int a, int b)**, welche rekursiv die Summe aus **a** und **b** berechnet und als Rückgabewert zurückliefert
3. Implementiere eine Methode **int subtract(int a, int b)**, welche rekursiv die Differenz aus **a** und **b** berechnet (**a - b**) und als Rückgabewert zurückliefert
4. Implementiere eine Methode **int multiply(int a, int b)**, welche rekursiv das Produkt von **a** und **b** berechnet und als Rückgabewert zurückliefert
5. Implementiere eine Methode **int divide(int a, int b)**, welche rekursiv den Quotienten von **a** und **b** berechnet (**a/b**; nur als natürliche Zahl) und als Rückgabewert zurückliefert

Hinweis: Hierfür muss **divide** überladen werden

Hinweis: Hierfür muss **multiply** überladen werden

Fibonacci-Folge

Die [Fibonacci-Sequenz](#) ist eine unendliche Zahlenreihe, die eine wichtige Bedeutung in der Mathematik hat. Sie beginnt mit zwei Einsen und jede weitere Zahl ergibt sich als die Summe der beiden vorangegangenen Zahlen. Die ersten Fibonacci-Zahlen sind wie folgt:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, [...]

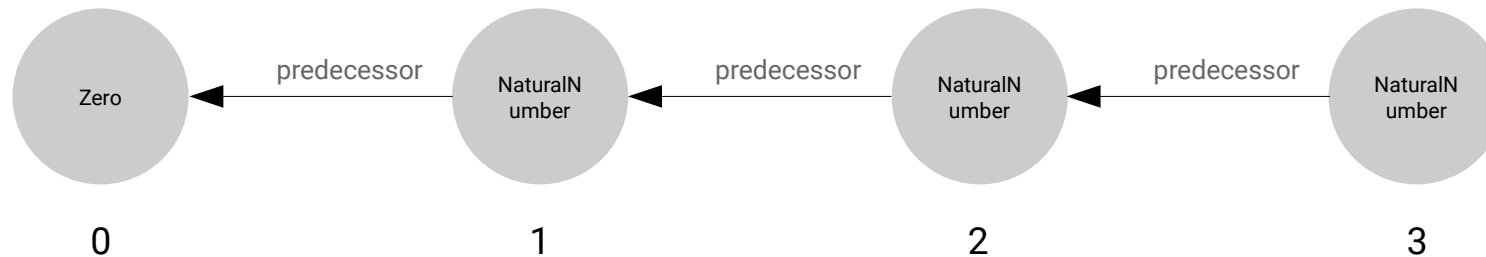
1. Erzeuge die Klasse **Fibonacci**, mit einer Main-Methode.
2. Entwerfe die Methode
`void printFibonacciSequence(int fiboPrevious, int fiboCurrent)`
Diese soll die Fibonacci-Sequenz auf der Konsole ausgeben
– bis zur ca. zwanzigsten Stelle.

Tipp: Summiere **fiboPrevious** und **fiboCurrent** zu **fiboNext** und übergebe beim Selbstaufruf **fiboCurrent** und **fiboNext**.
3. Erstelle eine neue Methode `int getFibonacciNumberAt(int position)`, welche die Fibonacci-Nummer an der angegebenen Position zurückgibt (z.B. die 15. Fibonacci-Nummer ist 610).

Tipp: Ermittle per Selbstaufruf die Fibonacci-Nummern an der vorherigen Position (P-1) und an der vor-vorherigen Position (P-2) und summiere beide. Für die Positionen 1 und 2 ist eine Sonderregelung (**if**) nötig. Verwende keine der zuvor programmierten Methoden – denn für diese letzte Aufgabe ist ein anderer Ansatz nötig.

Natural Numbers

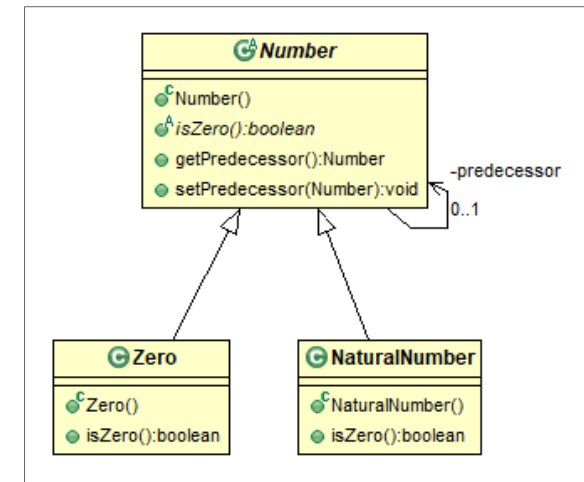
Natürliche Zahlen können als eine rekursive Datenstruktur betrachtet werden (als Abwandlung einer einfach-verketteten Liste):



1. Erzeuge die Klassen **Number**, **Zero** und **NaturalNumber** (siehe das Klassendiagramm als Beispiel)
2. Implementiere die Methode **isZero()** (abstrakt in **Number**), die **true** zurückgibt, wenn das Objekt von Typ **Zero** ist, ansonsten **false**
3. Die Klasse **Number** sollte ein Attribut haben mit Namen **predecessor** von Typ **Number** und auch die dazugehörigen Akzessor-Methoden
4. Implementiere die statischen Methoden **addition**, **subtraction**, **multiply** and **divide**. Jede dieser Methoden soll zwei Parameter von Typ **Number** entgegen nehmen und den Rückgabety **Number** haben

Die Signaturen sollten folgendermaßen aussehen:

```
public static Number addition(Number a, Number b)
public static Number subtraction(Number a, Number b)
public static Number multiply(Number a, Number b)
public static Number divide(Number a, Number b)
```



FileFinder

FileFinder soll die Festplatte nach Dateien durchsucht und diejenigen auflistet, die den definierten Filterkriterien entsprechen, z.B. Dateiname, Endung, Größe, Datum (aber alles der Reihe nach).

Hinweise: Dateisysteme sind hierarchisch aufgebaut – daher bietet sich hier ein rekursiver Algorithmus an.

Es soll die Java File-Klasse ([java.io.File](#)) verwendet werden, welche Teil der Standard-JCL ist.

Ein **File**-Objekt stellt eine einzelne Datei oder auch einen einzelnen Ordner im Dateisystem dar.

1. Erzeuge die Klasse **FileFinder**, mit einer Main-Methode.
2. Definiere den Root-Ordner für die Suche, z.B.:

```
// System Root
File rootFolder = new File("C:/");

// Eigene Dateien
File rootFolder = new File(System.getProperty("user.home"));
```
3. Definiere die rekursive Methode `findFilesRecursively(File parentFile)`.
4. Die Methode ermittelt die Liste aller Dateien/Unterordner die sich unmittelbar im übergebenen Ordner befinden:

```
File[] childFiles = parentFile.listFiles();
```
5. Die Methode iteriert über alle `childFiles` und gibt deren Pfad (`File.getPath()`) auf der Konsole aus.
6. Wenn eine `childFile` selbst ein Ordner ist (`isDirectory()`), dann ruft sich `findFilesRecursively()` selbst auf und übergibt dabei diese `childFile`. Dann geht es für diesen Unterordner wieder mit *Schritt 4* weiter.
7. Sonderfall: Wenn das Programm keine Zugriffsrechte auf einen Ordner hat, dann gibt `listFiles()` den Wert null zurück. Zwischen *Schritt 4* und *Schritt 5* sollte daher das `File`-Array auf null geprüft werden.
8. Filter 1: Gebe nur Dateien mit bestimmten Namen/Endungen auf der Konsole aus, z.B.:

```
if (childFile.getName().contains(".java")) {...}
```
9. Filter 2: Gebe nur Dateien mit einer bestimmten Größe auf der Konsole aus, z.B.:

```
if (childFile.length() > 10*1024 &&
    childFile.length() < 1024*1024) {...}
```
10. Filter 3: Gebe nur Dateien mit einem bestimmten Änderungsdatum auf der Konsole aus, z.B.:

```
if (childFile.lastModified() >
    (System.currentTimeMillis() - 1000*60*60*48)) {...}
```

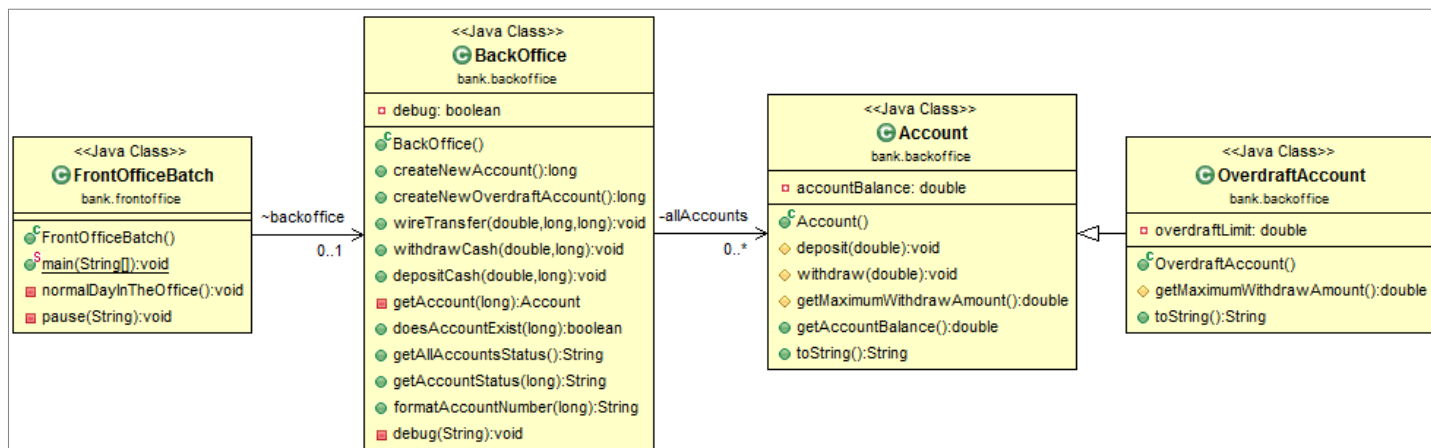
By the way, für professionelle Datumsoperationen sollte man auf die [Date-API](#) von Java 8 zurückgreifen.
11. Fortgeschritten: Sammle die Suchergebnisse, sortiere sie, und gebe sie erst am Ende aus (verwende Collections).

JavaBank

JavaBank soll eine Bank mit Konten, Überweisungen und Krediten simulieren.

1. Die Bank besteht aus einem **FrontOffice** (der Filiale), dem **BackOffice** (Zentrale mit Hochsicherheits-Rechenzentrum) sowie mehreren **Accounts** (Bankkonten).
2. Das **BackOffice** hat eine **Collection** aller Accounts und kann jeden Account über dessen eindeutige Kontonummer (**long**) identifizieren (z.B. durch die **Map: allAccounts<Long, Account>**).
3. Es existieren nur ein einziges **FrontOffice** sowie ein einziges **BackOffice**, jedoch mehrere Accounts.
4. Jeder Account hat eine **accountBalance** (den aktuellen Kontostand; der Einfachheit halber ein **double**).
5. Das **FrontOffice** hat aus Sicherheitsgründen keinen direkten Zugriff auf die **Account**-Objekte oder deren **accountBalance** und kann selbst keine **Account**-Objekte erzeugen. Jedoch hat das **FrontOffice** eine Verbindung (Referenz) zum **BackOffice** und kann bei diesem folgende sicherheitskritische Vorgänge anfordern:
 - Anlage eines neuen (leeren) Accounts:
BackOffice.createNewAccount()
Dies liefert dem FrontOffice die Kontonummer des neu angelegten Accounts zurück.
 - Bareinzahlung auf Account:
BackOffice.depositCash(amount, accountNr)
 - Barabhebung von Account:
BackOffice.withdrawCash(amount, accountNr)
 - Überweisung zw. Accounts:
BackOffice.wireTransfer(amount, fromAcctNr, toAcctNr)
 - Abhebungen und Überweisungen werden vom **BackOffice** nur bei ausreichendem Kontostand verarbeitet.
6. Um den Kontostand einsehen zu können, kann das **FrontOffice** folgende Methoden verwenden:
 - Zeige Kontostand eines Accounts auf Konsole:
BackOffice.getAccountStatus(accountNr)
 - Zeige Kontostände aller Accounts auf Konsole:
BackOffice.getAllAccountsStatus()
7. Das Programm beginnt in **FrontOffice.main()** und ruft dann **FrontOffice.normalDayInTheOffice()** auf. Dort werden mindestens 2 Accounts angelegt. Es gibt einige Bareinzahlungen, Abhebungen und Überweisungen. Ein paar Kundinnen und Kunden versuchen auch, mehr abzuheben/zu überweisen, als ihr Kontostand hergibt. Nach jeder Aktion werden alle Kontostände auf der Konsole angezeigt.
8. Zusatz: Der neue Kontotyp **OverdraftAccount** soll nun auch Dispokredite in begrenzter Höhe unterstützen.

Das Klassendiagramm könnte folgendermaßen aussehen:



Beispielhafte Ausgabe von `FrontOfficeBatch.normalDayInTheOffice()`:

```

Step 1: create two accounts
00000001 | Account | accountBalance=0.0
00000002 | OverdraftAccount | accountBalance=0.0 | overdraftLimit=500.0

-----

Step 2: customer deposits some cash in his account
00000001 | Account | accountBalance=0.0
00000002 | OverdraftAccount | accountBalance=1200.0 | overdraftLimit=500.0

-----

Step 3: customer wire-transfers some money to the webshop
00000001 | Account | accountBalance=1000.0
00000002 | OverdraftAccount | accountBalance=200.0 | overdraftLimit=500.0

-----

Step 4: customer wire-transfers some money to the webshop (now he's using the overdraft loan)
00000001 | Account | accountBalance=1400.0
00000002 | OverdraftAccount | accountBalance=-200.0 | overdraftLimit=500.0
    
```

JavaBank II

Ziel der Übung ist das Erweitern der JavaBank, um eine korrekte und sichere Behandlung von Fehlern.

1. Erweitere das bestehende **BackOffice** so, dass in folgenden Fällen Exceptions geworfen und korrekt behandelt werden. Der Begriff Banktransaktion umfasst hierbei Überweisungen, Bareinzahlungen und Barabhebungen.
 - Bei einer Barabhebung von einem normalen Konto übersteigt der Abhebungsbetrag den Kontostand.
 - Bei einer Barabhebung von einem Konto mit Disporahmen übersteigt der Abhebungsbetrag die Summe von Kontostand und Dispo.
 - Bei einer Barabhebung oder -einzahlung übersteigt der Betrag eine fest definierte Obergrenze (welche sich durch begrenzte Bargeldbestände in den Filialen/Automaten ergibt).
 - Bei einer Banktransaktion wird eine nicht existente Kontonummer verwendet.
 - Bei einer Überweisung sind Quell- und Zielkonto identisch.
 - Bei einer Banktransaktion wird ein negativer Betrag angegeben.
 - Bei einer Banktransaktion wird ein ungewöhnlich hoher Betrag angegeben (z.B. über 1 Million).
 - Bei einer Banktransaktion wird ein Betrag angegeben, der mehr als 2 Dezimalstellen hat.
2. In allen genannten Fällen darf die Banktransaktion nicht fortgesetzt werden. Stattdessen muss eine verständliche Fehlermeldung [`e.getMessage()`] auf der Konsole ausgegeben werden, jedoch soll nicht der Stacktrace angezeigt werden ([Information Leakage](#)). Alle Prüfungen müssen durchgeführt werden, bevor eine `accountBalance` verändert wird.
3. Disclaimer: Nun ist die JavaBank zwar sicherer, jedoch fehlen noch viele essenzielle Dinge wie z.B.:
 1. Persistenz ([ACID](#))
 2. Korrekte Währungs-Datentypen ([Currency](#))
 3. Sicherer nebenläufiger Zugriff ([Concurrency](#))
 4. Authentisierung und Autorisierung ([JAAS](#))
 5. KundInnen-Stammdatenverwaltung, Verzinsung, Besteuerung, Audit-Log, Archivierung
 6. Schnittstellen zu anderen Finanzinstituten (EBICS, SEPA, FIX, SWIFT, EDI, FinTech-APIs)
 7. Grafische Oberfläche (Filial/ATM-Client, Online/Mobile Banking)